PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification

For **TruthLayer**

(Dehallucination Add-On Module for Enhancing Reliability of AI Applications — Problem Statement #39)

Version 1.0 approved

Team details

Team Name	Project ID selected	Team Member 1 name and SRN per PES records	Team Member 2 name and SRN per PES records	Team Member 3 name and SRN per PES records	Team Member 4 name and SRN per PES records
NextGen	39	Shreyas M Mogaveera, PES2UG24AM219	Suhani SS , PES2UG24AM164	Usha Suresh Madival, PES2UG24AM175	CHINMAI A S, PES2UG24AM198

Prepared by: **Usha Suresh Madival, Shreyas M Mogaveera, Suhani SS, CHINMAI A S**

Department of Computer Science and Engineering (AI & ML) , PES University

05-09-2026

Table of Contents

Table of Contents 2
Revision History 3
1. Introduction 4
1.1 Purpose 4
1.2 Intended Audience and Reading Suggestions 4
1.3 Product Scope 4
1.4 References 4
2. Overall Description 5
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 5
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 Assumptions and Dependencies 6
3. External Interface Requirements 7
3.1 User Interfaces 7
3.2 Software Interfaces 7
3.3 Communications Interfaces 7
4. Analysis Models 8
5. System Features 10
5.1 Claim Extraction 10
5.2 Retrieval-Augmented Verification 10
5.3 Citation and Confidence Reporting 11
5.4 Verification API 11
5.5 Evaluation and Batch Reporting Dashboard 11
5.6 Optional Feedback Capture (Stretch Goal) 12
6. Other Nonfunctional Requirements 13
6.1 Performance Requirements 13
6.2 Safety Requirements 13
6.3 Security Requirements 13
6.4 Software Quality Attributes 13
6.5 Business Rules 13
7. Other Requirements 14
Appendix A: Glossary 15
Appendix B: Field Layouts 16
Appendix C: Requirement Traceability Matrix 17

Revision History

Name	Date	Reason For Changes	Version
NextGen	05/09/2026	Initial draft of SRS for TruthLayer (Dehallucination Add-On Module)	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for TruthLayer (Version 1.0), a Dehallucination Add-On Module built to satisfy Problem Statement #39: "Dehallucination Add-On Module for Enhancing Reliability of AI Applications." TruthLayer is a middleware module that intercepts text generated by any Large Language Model (LLM) or chatbot, decomposes it into verifiable factual claims, cross-references those claims against a trusted knowledge base using retrieval-augmented verification and Natural Language Inference (NLI), and returns an annotated response indicating which claims are supported, contradicted, or unverifiable. This document covers the complete scope of the module — the AI/ML verification pipeline, the backend API, and the demonstration frontend — as a single integrated system for the 8-week Software Engineering mini-project (Subject Code UE24CS341A).

1.2 Intended Audience and Reading Suggestions

This document is intended for: (a) the 4-member development team, to guide iterative Agile implementation and division of work; (b) the course instructor/evaluator, to assess requirement completeness, testability, and traceability during weekly and final evaluations; (c) any future maintainer extending the module. Readers unfamiliar with hallucination-detection terminology should first review Appendix A (Glossary). Section 5 (System Features) is organized by functional module and is the primary reference during implementation and viva.

1.3 Product Scope

TruthLayer is a pluggable reliability layer, not a standalone chatbot. Its objective is to reduce the real-world risk of LLM outputs containing fabricated or unverifiable facts, a growing concern for any team deploying customer-facing or research-facing AI applications. The product exposes a REST API (POST /verify) that any chatbot, content generator, or internal tool can call with generated text; it responds with the text annotated at claim-level with a verdict (Supported / Contradicted / Unverifiable), a confidence score, and supporting citations drawn from a configurable knowledge base (default: a Wikipedia text subset). A web dashboard demonstrates the pipeline interactively and reports aggregate hallucination-rate metrics against published benchmarks (HaluEval, TruthfulQA). The product does not generate LLM text itself and does not guarantee 100% detection accuracy — its goal is measurable, explainable risk reduction, not infallibility.

1.4 References

- Li, J. et al. (2023). HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. EMNLP 2023. https://github.com/RUCAIBox/HaluEval
- Lin, S. et al. (2022). TruthfulQA: Measuring How Models Mimic Human Falsehoods. https://huggingface.co/datasets/truthfulqa/truthful_qa
- Thorne, J. et al. (2018). FEVER: a large-scale dataset for Fact Extraction and VERification. https://fever.ai/
- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications (template basis for this document).
- PES University, Dept. of CSE — Jackfruit Miniproject Software Engineering Project Guidelines, Subject Code UE24CS341A.

2. Overall Description

2.1 Product Perspective

TruthLayer is a new, self-contained middleware product. It is not a replacement for an existing university system and has no legacy predecessor. It sits logically between an LLM-based application (the "source" system, e.g. a support chatbot) and the end user, intercepting generated text before or after it reaches the user. It is designed to be deployed either as (a) a synchronous API call from a third-party chatbot backend, or (b) a standalone demo application where a user pastes or generates text directly against the module. The system comprises four major components: the Claim Extraction module, the Retrieval-Augmented Verification module, the Backend API/orchestration layer, and the Frontend visualization dashboard.

2.2 Product Functions

At a high level, TruthLayer allows a user (or an integrated application) to:

- Submit a block of AI-generated text for verification via the API or the web UI.
- Receive the text decomposed into individual factual claims.
- Receive a verdict per claim — Supported, Contradicted, or Unverifiable — with a numeric confidence score.
- Receive supporting evidence citations (source passage + reference) for each Supported/Contradicted claim.
- Swap or extend the underlying knowledge base used for verification (e.g., a custom document corpus).
- View an aggregate evaluation report (precision, recall, F1) of the pipeline's performance against benchmark datasets (HaluEval, TruthfulQA).
- Export a batch verification report (multiple text samples processed together) as a downloadable summary.

2.3 User Classes and Characteristics

User Class	Frequency / Expertise	Characteristics
Developer Integrator	Frequent; technical (knows REST APIs)	Calls the /verify endpoint from their own chatbot/content-generation backend; primary consumer of the API and API documentation.
Evaluator / End Demo User	Occasional; non-technical	Uses the web dashboard to paste text and visually inspect the annotated, colour-coded verdicts; needs no technical background.
ML/Research User	Occasional; technical	Uses the evaluation dashboard to review precision/recall metrics of the verification pipeline against benchmark datasets.

2.4 Operating Environment

Backend: Python 3.11, FastAPI, running on Linux (Ubuntu 22.04/24.04) — CPU-only inference is sufficient (no dedicated GPU is a hard requirement). Frontend: React/Next.js, served via any modern browser (Chrome, Firefox, Edge — last two major versions). Containerized via Docker for consistent deployment across development machines. Vector store: Chroma (embedded, file-backed) for the knowledge base index. Development machines: any laptop/desktop with a minimum of 8 GB RAM; a GPU/NPU (e.g., Intel integrated NPU via OpenVINO) is an optional performance optimization, not a requirement.

2.5 Design and Implementation Constraints

- Must be built primarily using free/open-source tools and pretrained models (no paid API dependency for the core pipeline).
- Must run acceptably on CPU-only hardware within the team's available machines (8-week academic timeline; no cloud GPU budget).

- Must follow Agile methodology with GitHub-tracked user stories/backlog, as mandated by the UE24CS341A course guidelines.
- Must expose a documented REST API so the module can integrate with any third-party chatbot without modification to that chatbot's internals.
- Knowledge base for the MVP is restricted to a defined Wikipedia text subset (full open-domain web coverage is out of scope for 8 weeks).

2.6 Assumptions and Dependencies

- Assumes availability of pretrained NLI and embedding models from HuggingFace (e.g., a distilled MNLI model, sentence-transformers) with no internet access required at inference time once downloaded.
- Assumes the HaluEval and TruthfulQA benchmark datasets remain publicly accessible for evaluation throughout the project timeline.
- Assumes team access to a development machine with a minimum of 8 GB RAM and ≥ 10 GB free disk space for models and the knowledge-base index.
- Depends on third-party open-source libraries (HuggingFace Transformers, sentence-transformers, FastAPI, Chroma) remaining stable/compatible for the project duration.
- Assumes the team can manually curate a held-out, ground-truth-labeled sample of at least 150 claims drawn from the benchmark datasets (Section 1.4) to support the accuracy target in REQ-19 and the evaluation dashboard (Section 5.5).

3. External Interface Requirements

3.1 User Interfaces

The web dashboard provides: (a) a text-input panel where a user pastes or types AI-generated text; (b) a "Verify" action button; (c) an annotated output view where each claim is highlighted — green for Supported, red for Contradicted, amber for Unverifiable — with an expandable citation panel per claim; (d) an evaluation tab showing precision/recall/F1 charts computed against benchmark data; (e) a batch-upload tab for processing multiple text samples and exporting a summary report. Standard elements (navigation bar, help tooltip, loading indicator, error banner) appear consistently across all views. No login/registration screen is required for the MVP demo; an API key field is provided for the developer-integrator flow only.

3.2 Software Interfaces

Component	Technology / Version	Purpose
NLI Verification Model	HuggingFace Transformers — e.g. cross-encoder/nli-deberta-v3-small	Determines entailment/contradiction/neutral relationship between a claim and retrieved evidence.
Embedding Model	sentence-transformers/all-MiniLM-L6-v2	Generates dense vector embeddings for semantic retrieval.
Vector Store	Chroma (embedded)	Stores and retrieves knowledge-base passage embeddings.
Backend Framework	FastAPI (Python 3.11)	Exposes REST endpoints, orchestrates the verification pipeline.
Relational Database	PostgreSQL 15	Stores usage logs, batch job records, and evaluation run history.
Frontend Framework	React (Next.js)	Renders the dashboard and annotated-text UI.

3.3 Communications Interfaces

All communication between the frontend and backend, and between an integrating third-party chatbot and TruthLayer, occurs over HTTPS using JSON request/response payloads (REST, HTTP/1.1). The primary endpoint is POST /verify, accepting a JSON body with a text field (and optional knowledge_base_id field) and returning a JSON response containing the list of extracted claims, verdicts, confidence scores, and citations. Standard HTTP status codes are used for error signalling (400 for malformed input, 401 for missing/invalid API key, 500 for internal pipeline failure). No real-time/streaming protocol is required for the MVP; a future version may expose a WebSocket for streaming verification of long-form text (see Section 7).

4. Analysis Models

The primary use cases are summarised below in place of a graphical use-case diagram (to be elaborated with a UML diagram in the High-Level Architecture document, per Jackfruit Phase-2 deliverables):

Use Case	Actor	Description
UC-1: Verify Text	Developer Integrator / Evaluator	Actor submits AI-generated text; system returns claim-level verdicts with citations.
UC-2: Swap Knowledge Base	Developer Integrator	Actor uploads/points to a custom corpus; system re-indexes and uses it for subsequent verification.
UC-3: Run Benchmark Evaluation	ML/Research User	Actor triggers an evaluation run against HaluEval/TruthfulQA; system reports precision/recall/F1.
UC-4: Batch Verify	Developer Integrator	Actor submits multiple text samples; system processes them and returns a downloadable summary report.

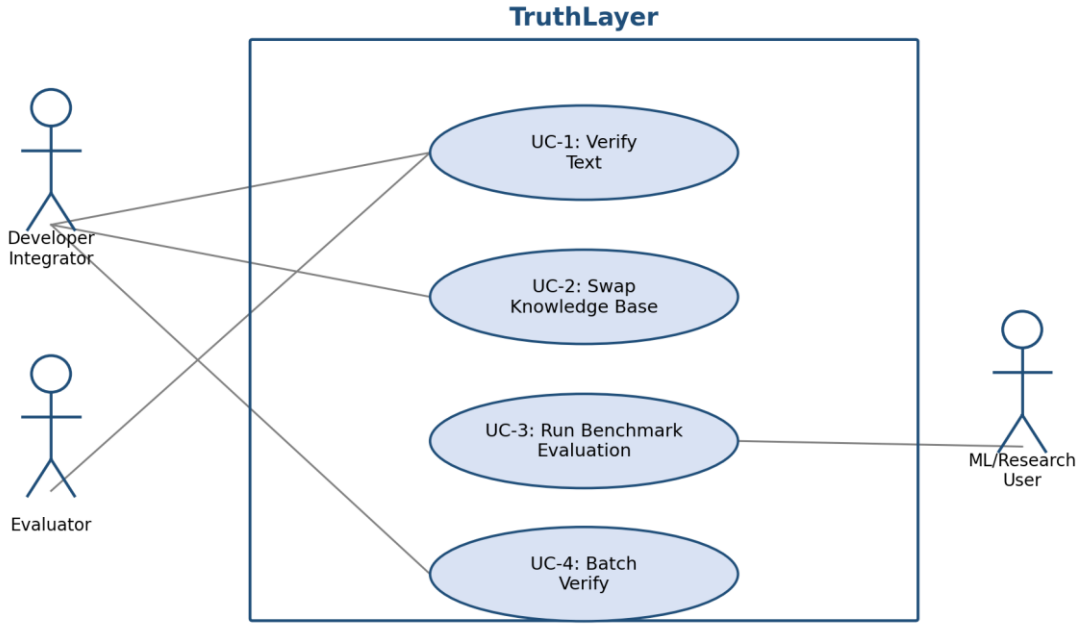


Figure 1: Use Case Diagram — TruthLayer

Data flow (high level): AI-generated text → Claim Extraction → [per claim] Dense Retrieval over Knowledge Base → NLI Verification → Confidence Calibration → Annotated Response Assembly → API Response / Dashboard Rendering. A corresponding entity-relationship model (VerificationRequest, Claim, Evidence, EvaluationRun) will be detailed in the Software Design Document (Jackfruit Phase-2).

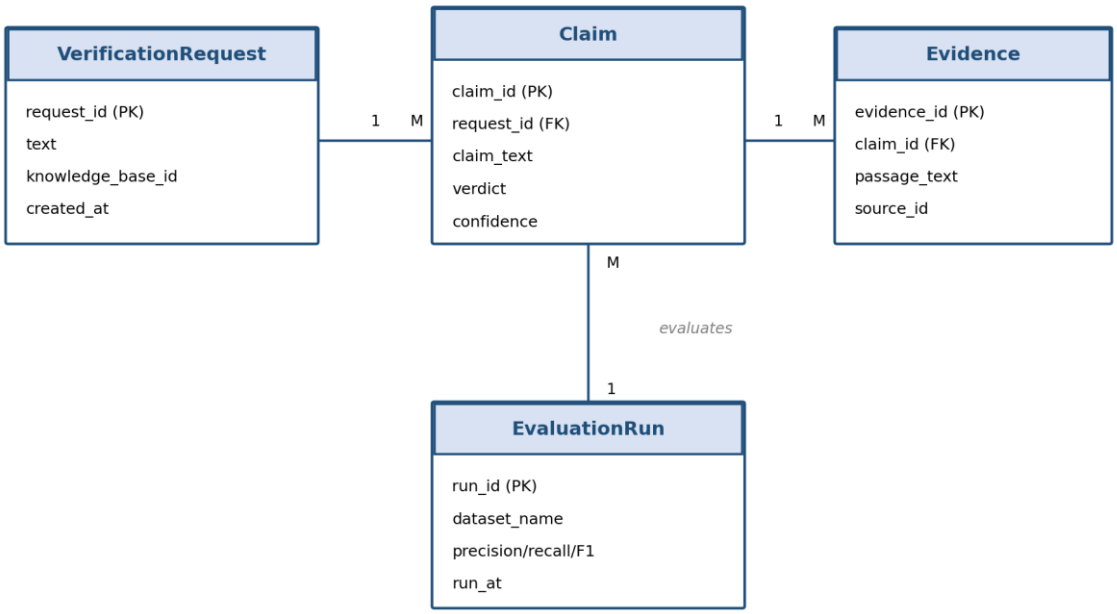


Figure 2: Entity-Relationship Diagram — TruthLayer core entities

5. System Features

5.1 Claim Extraction

5.1.1 Description and Priority

Decomposes a block of AI-generated text into a list of discrete, checkable factual claims (atomic statements). Priority: High (benefit: 9, cost: 5, risk: 4).

5.1.2 Stimulus/Response Sequences

User/integrator submits raw text → system returns an ordered list of extracted claim strings, each linked back to its source sentence span.

5.1.3 Functional Requirements

REQ-1: The system shall accept a text input of up to 2,000 words for claim extraction.

REQ-2: The system shall decompose the input into atomic factual claims, excluding subjective opinions and rhetorical statements.

REQ-3: The system shall preserve a mapping from each extracted claim back to its original sentence span in the source text.

REQ-4: If no checkable factual claims are found in the input, the system shall return an explicit "No verifiable claims found" result rather than an error.

5.2 Retrieval-Augmented Verification

5.2.1 Description and Priority

For each extracted claim, retrieves relevant evidence passages from the configured knowledge base and determines whether the evidence supports, contradicts, or is insufficient to judge the claim. Priority: High (benefit: 9, cost: 7, risk: 6).

5.2.2 Stimulus/Response Sequences

Claim list generated (from 5.1) → system retrieves top-k evidence passages per claim → system returns a verdict (Supported / Contradicted / Unverifiable) and confidence score per claim.

5.2.3 Functional Requirements

REQ-5: The system shall retrieve the top-k (default k=5) most semantically relevant passages from the knowledge base for each claim.

REQ-6: The system shall classify each claim as Supported, Contradicted, or Unverifiable using an NLI model applied to the claim and its retrieved evidence.

REQ-7: The system shall attach a numeric confidence score (0.0–1.0) to every verdict.

REQ-8: The system shall label a claim as Unverifiable when no retrieved passage exceeds a configurable minimum relevance threshold.

REQ-9: The system shall support swapping the knowledge base (default: Wikipedia subset) for a custom document corpus supplied by the integrator.

REQ-19: The system shall achieve an overall verification accuracy of at least 80% (correct Supported/Contradicted/Unverifiable classification) when evaluated against a held-out, manually labeled sample of at least 150 claims (see Section 2.6).

5.3 Citation and Confidence Reporting

5.3.1 Description and Priority

Attaches the specific evidence passage(s) and source reference used to reach each verdict, and calibrates the confidence score against known evaluation benchmarks. Priority: Medium (benefit: 7, cost: 4, risk: 3).

5.3.2 Stimulus/Response Sequences

Verdict generated (from 5.2) → system attaches the exact evidence text and source identifier → annotated response is returned to the caller.

5.3.3 Functional Requirements

REQ-10: The system shall return, for every Supported or Contradicted claim, the exact evidence passage text and its source identifier.

REQ-11: The system shall calibrate raw model confidence scores so that a score of 0.9 corresponds empirically to ≥ 90% correctness on a held-out benchmark sample.

5.4 Verification API

5.4.1 Description and Priority

Exposes the entire pipeline as a documented REST API so that any third-party chatbot or content generator can integrate TruthLayer without modification to its own internals. Priority: High (benefit: 9, cost: 6, risk: 4).

5.4.2 Stimulus/Response Sequences

Integrator sends POST /verify with a JSON payload → system processes the pipeline end-to-end → system returns a JSON response within the specified latency bound (see Section 6.1).

5.4.3 Functional Requirements

REQ-12: The system shall expose a POST /verify endpoint accepting JSON input and returning a JSON response conforming to the schema in Appendix B.

REQ-13: The system shall require a valid API key on every request and reject unauthenticated requests with HTTP 401.

REQ-14: The system shall log every request (timestamp, input length, processing time, verdict summary) for later evaluation and debugging.

REQ-15: The system shall return HTTP 400 with a descriptive error message when the input payload is malformed or exceeds the accepted length.

5.5 Evaluation and Batch Reporting Dashboard

5.5.1 Description and Priority

Provides a visual interface for interactive single-text verification, benchmark evaluation, and batch processing of multiple text samples with an exportable summary report. Priority: Medium (benefit: 8, cost: 5, risk: 3).

5.5.2 Stimulus/Response Sequences

User pastes text and clicks Verify → dashboard displays colour-coded annotated output with expandable citations. User triggers a benchmark run → dashboard displays precision/recall/F1 charts. User uploads a batch file → dashboard displays aggregate results and offers a downloadable report.

5.5.3 Functional Requirements

REQ-16: The system shall visually distinguish Supported, Contradicted, and Unverifiable claims using distinct colour coding in the dashboard.

REQ-17: The system shall display precision, recall, and F1 score computed against a held-out sample of HaluEval and/or TruthfulQA.

REQ-18: The system shall accept a batch of up to 50 text samples in a single upload and return a consolidated, downloadable report (PDF or CSV).

5.6 Optional Feedback Capture (Stretch Goal)

5.6.1 Description and Priority

Allows a dashboard user to record lightweight Agree/Disagree feedback on a displayed verdict, aggregated for later review. Priority: Low / Stretch (V1.1) — not required for the V1.0 MVP demo or grading; included to show roadmap awareness without expanding the core scope (no authentication or per-user account system is introduced for this feature). Benefit: 5, cost: 3, risk: 2.

5.6.2 Stimulus/Response Sequences

User views a verdict on the dashboard and clicks Agree or Disagree → system increments an aggregate counter for that verdict category → updated aggregate agreement rate is shown on the Evaluation tab (Section 5.5).

5.6.3 Functional Requirements

REQ-20 (Stretch): If implemented, the system shall allow any dashboard user to submit a single Agree/Disagree reaction on a displayed verdict, with no login or API key required.

REQ-21 (Stretch): If implemented, feedback shall be stored and displayed only as an aggregate agreement rate per verdict category, not tied to an individual user identity, to avoid introducing an authentication/account subsystem.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- NFR-1: The /verify endpoint shall return a response within 5 seconds for an input of up to 200 words, on the reference CPU-only development hardware.
- NFR-2: The system shall support at least 10 concurrent verification requests without failure (queued, not necessarily parallel, for the MVP).
- NFR-3: Knowledge-base retrieval (top-k passage lookup) shall complete within 500 ms per claim.

6.2 Safety Requirements

- NFR-4: The system shall never present an Unverifiable claim as Supported; when evidence is insufficient, the system shall default to Unverifiable rather than guessing.
- NFR-5: All dashboard outputs shall carry a visible disclaimer that TruthLayer reduces, but does not eliminate, the risk of hallucinated content, and is not a substitute for human review in high-stakes use cases.

6.3 Security Requirements

- NFR-6: All API endpoints shall require a valid API key; keys shall be stored hashed, never in plaintext.
- NFR-7: The system shall not persist submitted text beyond the retention period required for the current session's logging/evaluation (no indefinite storage of third-party user content).
- NFR-8: All communications between frontend, backend, and any integrating client shall use HTTPS/TLS in the deployed environment.

6.4 Software Quality Attributes

- Maintainability: the Claim Extraction, Retrieval, NLI Verification, and API layers shall be independently replaceable modules with documented interfaces, verified via unit tests per module.
- Portability: the system shall run identically via Docker Compose on any Linux/macOS/Windows (WSL2) development machine.
- Reliability: the system shall degrade gracefully — if the NLI model fails to load, the system shall fall back to embedding-similarity-only verification (see Section 7) rather than crash.
- Usability: a first-time user shall be able to complete a single text verification within 2 minutes of opening the dashboard, without external instructions.

6.5 Business Rules

- Only an authenticated Developer Integrator may swap or upload a custom knowledge base.
- Evaluation/benchmark runs are read-only against fixed benchmark datasets and do not modify the production knowledge base.
- Domain requirement: the default knowledge base (Wikipedia subset) must be refreshed/re-indexed at least once during the project to demonstrate the update workflow.

7. Other Requirements

- Database Requirement: PostgreSQL shall be used to persist API usage logs, batch job metadata, and evaluation run history; the vector knowledge-base index is stored separately in Chroma.
- Internationalization: out of scope for v1.0 — the MVP supports English-language text only.
- Legal Requirement: the system shall not store or transmit personally identifiable information; all demo/benchmark data used is publicly licensed (HaluEval, TruthfulQA, FEVER, Wikipedia).
- Fallback Requirement: if the primary NLI verification model is unavailable or underperforms, the system shall fall back to a simpler embedding-similarity-threshold verification method, preserving the same API contract (see Part 18 risk analysis of the project proposal).
- Reuse Objective: pretrained models (NLI, embeddings) and public benchmark datasets shall be reused as-is wherever possible; only lightweight glue code and, optionally, a calibration layer, will be newly developed.

Appendix A: Glossary

Term	Definition
Hallucination	A factually incorrect or fabricated statement generated by an AI language model that is presented as if true.
Claim	An atomic, checkable factual statement extracted from a larger block of AI-generated text.
NLI (Natural Language Inference)	An NLP task/model that determines whether a hypothesis (claim) is entailed, contradicted, or neutral with respect to a premise (evidence).
RAG (Retrieval-Augmented Generation/Verification)	A technique that retrieves relevant external evidence to ground or verify a model's output.
Knowledge Base (KB)	The configurable corpus of trusted documents (default: Wikipedia subset) against which claims are verified.
Embedding	A dense numeric vector representation of text used for semantic similarity search.
Confidence Score	A calibrated numeric value (0.0–1.0) indicating the system's certainty in a given verdict.
Verdict	The classification outcome for a claim: Supported, Contradicted, or Unverifiable.
HaluEval / TruthfulQA / FEVER	Public benchmark datasets used to train-test and evaluate hallucination-detection performance.

Appendix B: Field Layouts

POST /verify — Request Schema

Field	Length	Data Type	Description / Is Mandatory
text	≤ 2000 words	String	The AI-generated text to be verified. Mandatory: Y
knowledge_base_id	≤ 64 chars	String	Identifier of the knowledge base to verify against; defaults to "wikipedia_subset" if omitted. Mandatory: N
api_key	32	Alphanumeric	Caller's authentication key, supplied via request header. Mandatory: Y

POST /verify — Response Schema

Field	Length	Data Type	Description
claims	variable	Array<Object>	List of extracted claims, each with text, span, verdict, confidence, and citations.
claims[].verdict	≤ 15 chars	Enum String	One of: Supported | Contradicted | Unverifiable.
claims[].confidence	3	Float (0–1)	Calibrated confidence score for the verdict.
claims[].citations	variable	Array<Object>	Evidence passage text + source identifier supporting the verdict.
processing_time_ms	6	Integer	Server-side processing time in milliseconds.

Evaluation Report — Fields Included

Benchmark Evaluation Report	Batch Verification Report
Dataset name (HaluEval / TruthfulQA)	Batch ID / timestamp
Sample size	Number of text samples processed
Precision / Recall / F1	Per-sample verdict counts (Supported/Contradicted/Unverifiable)
Confusion matrix summary	Aggregate hallucination rate
Average processing latency	Export format (PDF/CSV) and download link

Appendix C: Requirement Traceability Matrix

Sl.No	Req ID	Brief Description	Architecture Ref.	Design Ref.	Code File Ref.	Test Case ID
1	REQ-1	Accept text input for claim extraction	TBD	TBD	TBD	UT-01
2	REQ-2	Decompose text into atomic factual claims	TBD	TBD	TBD	UT-02
3	REQ-5	Retrieve top-k evidence passages per claim	TBD	TBD	TBD	UT-05
4	REQ-6	Classify claim verdict via NLI	TBD	TBD	TBD	UT-06
5	REQ-9	Support custom knowledge base swap	TBD	TBD	TBD	IT-01
6	REQ-12	Expose POST /verify REST endpoint	TBD	TBD	TBD	IT-02
7	REQ-13	Require valid API key on every request	TBD	TBD	TBD	UT-13
8	REQ-16	Colour-coded verdict display	TBD	TBD	TBD	ST-01
9	REQ-17	Display benchmark precision/recall/F1	TBD	TBD	TBD	ST-02
10	REQ-18	Batch verification with downloadable report	TBD	TBD	TBD	ST-03